



التقرير النهائي

مترجم لتطبيق ويب بلغتي Python/Flask و Jinja2

مشروع مادة المترجمات

إعداد الطلاب

1. علي أحمد يزبك
2. زينب هيثم محمد
3. رامة مهند أبو العينين
4. علي سليمان سليمان
5. سارة عبد القادر النبهان

31

رقم المجموعة

م. آيه شحادة ٠ م. سارة المقبعة

بإشراف

github.com/aliyazbek/compiler-project-yaz

مستودع المشروع

2025 / 2026

الفهرس

1	مقدمة وبنية المشروع	نظرة عامة · المراحل · البناء والتشغيل
2	التحليل اللفظي والنحوي	المطلب الأول
3	بناء الشجرتين والتابع المولّد	المطلب الثاني
4	بنية العقد ومخطط الشجرة	المطلب الثالث
5	التحليل الدلالي	المطلب الرابع
6	توليد الكود	المطلب الخامس
7	الواجهات والتنقل	المطلب السادس
8	الطباعة وجدول الرموز	المطلب السابع
9	الخاتمة ونطاق المترجم	الحصيلة · آفاق التطوير

ملخص تنفيذي

يعرض هذا التقرير مترجماً مكتوباً بلغة Java يعالج لغتين مصدريتين لتطبيق ويب مبني بإطار العمل Flask، وينفذ المراحل الكلاسيكية كاملة: التحليل اللفظي، فالنحوي، فبناء شجرتي القواعد المجردة، فجدول الرموز، فالتحليل الدلالي، ثم توليد موقع قابل للعمل. بُني المشروع على ANTLR 4.13.2 بأربعة ملفات قواعد كُتبت يدوياً، ويقع في 72 ملف Java (8,639 سطرًا) تغطّيها 64 اختباراً آلياً.

المؤشر	القيمة	المؤشر	القيمة
ملفات القواعد	4	عقد الشجرتين	707
أنواع الرموز اللفظية	97	الفحوص الدلالية	9
القواعد النحوية	52	الصفحات المولّدة	5
أصناف العقد	37	الاختبارات الناجحة	64 / 64

1.1 فكرة المشروع

المشروع مترجم مكتوب بلغة Java يعالج تطبيق ويب مبنياً بإطار العمل Flask. يتألف التطبيق من لغتين مصدريتين منفصلتين، لكل واحدة منهما قواعد نحوية مستقلة وشجرة تجريدية خاصة بها:

الجزء	اللغة المصدرية	الملفات المصدرية
الخلفي (Backend)	Python / Flask	test_programs/run_app.py
الأمامي (Frontend)	HTML + CSS + Jinja2	static/style.css · templates/*.html

ينفذ المترجم المراحل الكلاسيكية كاملة – التحليل اللفظي، فالنحوي، فبناء شجرة القواعد المجردة، فجدول الرموز، فالتحليل الدلالي – ثم يولد من الشجرتين موقعاً قابلاً للعمل. بُني على **ANTLR 4.13.2** المرفقة ضمن المجلد `dependencies/`، فلا يحتاج البناء إلى اتصال بالإنترنت.

1.2 مراحل المترجم

يطبع البرنامج الرئيسي `app.Main` ست مراحل مرقّمة بالترتيب التالي:

المرحلة	العنوان المطبوع	الوظيفة
1	BACKEND COMPILATION (Python / Flask)	ترجمة <code>run_app.py</code>
2	TEMPLATE COMPILATION (HTML + Jinja2)	ترجمة القوالب الستة
3	STYLESHEET COMPILATION (CSS)	ترجمة <code>style.css</code>
4	GENERATOR (Python data array → Jinja tree)	تمرير البيانات إلى الشجرة الثانية
5	CODE GENERATION (HTML output)	توليد الصفحات والملفات
6	SEMANTIC ANALYSIS (error checking)	فحص الأخطاء الدلالية

ثم يطبع قسمين إضافيين: **جدول الرموز** مع شجرة النطاقات، و**شجرة الـ AST** للملف الخلفي، وينتهي بالسطر:

```
ALL PHASES PASSED - compilation succeeded with 0 errors.
```

1.3 بنية الشيفرة المصدرية

يتوزع المشروع على اثنتي عشرة حزمة (Package) مكتوبة يدوياً:

الحزمة	الملفات	الدور
app	2	نقطتا الدخول: <code>Main</code> و <code>Serve</code>
compiler	2	خط المعالجة الموحد و <code>CompilationResult</code>
visitor	2	تحويل شجرة الاشتقاق إلى AST

الحزمة	الملفات	الدور
ast/base backend frontend	37	أصناف العقد: الأساس المجرد وعقد Python وعقد HTML/Jinja/CSS
symboltable	3	SymbolTable · Scope · Symbol
semantic	2	Diagnostic · SemanticAnalyzer
generator	4	HtmlEmitter · TemplateContext · ContextBuilder · Generator
output	4	JSON · OutputWriter · BuildDriver
server	4	SourceWatcher · SiteRenderer · DataStore · AppServer
test	12	مجموعة الاختبارات

إحصاءات المشروع	الملفات	الأسطر
المترجم نفسه (Java)	60	6,970
الاختبارات الآلية – 64 اختباراً ناجحاً	12	1,669
ملفات القواعد .g4	4	475

1.4 خط المعالجة ونقطتا الدخول

عُرِّف خط المعالجة مرة واحدة في الصنف `compiler.Compiler` على الشكل: محلِّل لفظي ← محلِّل نحوي ← زائر بناء الشجرة ← جدول الرموز. ويُعيد كائن `CompilationResult` بدل تعديل حالة ساكنة. كاشفاً ثلاثة توابع عامة: `compileBackend()` و `compileFrontend()` و `compileCss()`. ويستهلكه برنامجان:

app.Main	يترجم مرة واحدة. يطبع كل المراحل. يكتب <code>output/</code> و <code>compiler_output/</code> . ثم ينتهي
app.Serve	يبقى قيد التشغيل: يعرض الموقع على المنفذ 8080 ويعيد توليده عند تغيّر البيانات أو الملفات

1.5 التطبيق النموذجي

الملف `test_programs/run_app.py` – واحد وخمسون سطرًا – يصرّح عن **ستة مسارات (Routes)** فوق ست دوال عرض، ويحتوي على مصفوفة بيانات واحدة اسمها `products` مؤلفة من ثلاثة قواميس. لكلٍّ منها المفاتيح الأربعة `id` و `name` و `image` و `details`.

1.6 البناء والتشغيل

المتطلّب الوحيد هو **JDK 17 أو أحدث**: ولا حاجة إلى أي تنزيل خارجي.

<code>./build.sh</code>	<code># build and run</code>	(Linux / macOS / Git Bash)
<code>./build.sh --serve</code>	<code># start the server on port 8080</code>	
<code>./build.sh --test</code>	<code># run the test suite</code>	
<code>.\build.ps1</code>	<code># build and run</code>	(Windows PowerShell)
<code>.\build.ps1 -Serve</code>	<code># start the server</code>	

المطلب الأول: بناء قواعد اللغة (Lexer & Parser) وتعريف Syntax الخاص بـ Python و Jinja2 و HTML و CSS.

2.1 ملفات القواعد

كُتبت أربعة ملفات قواعد بلغة ANTLR، بمجموع 475 سطراً:

الملف	الأسطر	الدور
PythonFlaskLexer.g4	89	الرموز اللفظية للغة Python
PythonFlaskParser.g4	128	القواعد النحوية للغة Python
FrontendLexer.g4	87	الرموز اللفظية لـ HTML + CSS + Jinja2
FrontendParser.g4	171	القواعد النحوية للثلاثة معاً

اعتمد في الجزء الخلفي **الفصل** بين المحلل اللفظي والنحوي في ملفين، بينما وُجِدَ الجزء الأمامي في زوج واحد يخدم اللغات الثلاث – لأن Jinja2 و CSS **مضمّناتان داخل** نصّ HTML، فلا يمكن تحليل أيّ منها بمعزل عن الأخرى.

2.2 اللغة الأولى: Python

التحليل اللفظي

يصرّح PythonFlaskLexer.g4 عن **73 قاعدة لفظية** تُنتج **45 نوع رمز (Token Type)** موزّعة على الفئات التالية:

الفئة	العدد	الرموز
كلمات مفتاحية	15	FROM IMPORT DEF RETURN IF ELIF ELSE FOR IN AND OR NOT TRUE FALSE NONE
عوامل	11	EQ NEQ LTE GTE ASSIGN PLUS MINUS STAR SLASH LT GT
رموز بنوية	10	LPAREN RPAREN LBRACKET RBRACKET LBRACE RBRACE COMMA COLON DOT AT
ثوابت ومعرّفات	4	STRING FLOAT INTEGER IDENTIFIER
تنسيق	3	NEWLINE · WS (مُهْمَل) · COMMENT (مُهْمَل)

التحليل النحوي

يعرّف PythonFlaskParser.g4 أربعاً وعشرين قاعدة نحوية، قاعدة البداية فيها program :

```
program · statement · importStatement · importList · assignmentStatement
functionDefinition · parameterList · parameter · functionBody
decoratedFunction · decorator · ifStatement · elifStatement · elseStatement
forStatement · returnStatement · expressionStatement · expression · primary
listLiteral · dictLiteral · keyValuePair · argumentList · argument
```

أسبقية العوامل مُشَقَّرة في ترتيب البدائل اليسارية العودية داخل القاعدة `expression`، من الأعلى أسبقيةً إلى الأدنى:

member access → index → call → not → * / → + - → < > <= >= → == != → and → or

وتحمل القواعد **23 بديلاً معنوياً (Labelled Alternative)** بمجموعها، ما يجعل كل شكل تعبير يوصل إلى تابع زيارة مستقل عند بناء الشجرة.

2.3 نمط المحلل اللفظي INSIDE_BRACKETS

في Python يُعدّ رمز نهاية السطر `NEWLINE` **فاصلاً بين التعليمات**، ولذلك يجب أن يبقى رمزاً ذا معنى. لكن Python تسمح في الوقت نفسه بامتداد التعبير على عدة أسطر ما دام داخل قوسين – وهي خاصية **الوصل الضمني للأسطر (Implicit Line Joining)** – كما في مصفوفة البيانات الممتدة على خمسة أسطر في `run_app.py`.

عولجت هذه الازدواجية بنمط لفظي (Lexer Mode) خاص: عند مصادفة `(` أو `[` أو `{` يدخل المحلل نمط `INSIDE_BRACKETS` عبر `pushMode`، وداخل هذا النمط تُهمل نهايات الأسطر تماماً:

```
// DEFAULT_MODE: a newline is a meaningful token
NEWLINE : ('\r'? '\n') ;
LBRACKET : '[' -> pushMode(INSIDE_BRACKETS) ;

mode INSIDE_BRACKETS;
IB_WS : [ \t\r\n]+ -> skip ; // newlines vanish inside brackets
IB_RBRACKET : ']' -> type(RBRACKET), popMode ;
```

ويُغادر النمط عند القوس المغلق عبر `popMode`. أما التداخل فمُعالج لأن النمط يُعيد التصريح عن الأقواس المفتوحة بـ `pushMode` مرة أخرى. ومن قواعد النمط الثلاثين تُعاد **ثمان وعشرون قاعدة** إلى أنواع النمط الافتراضي عبر `-> type(...)`، فلا يرى المحلل النحوي فرقاً بين رمز كُتب داخل الأقواس ورمز كُتب خارجها.

تحقق عملي. تحليل مصفوفة ممتدة على أربعة أسطر يُنتج **17 رمزاً** فيها رمز `NEWLINE` **واحد فقط** – وهو الذي يلي القوس المغلق. أما نهايات الأسطر الثلاث الداخلية فلا تُنتج أي رمز.

2.4 اللغات الثلاث الأخرى: HTML و CSS و Jinja2

يصرّح `FrontendLexer.g4` عن **60 قاعدة لفظية** تُنتج **52 نوع رمز**، ويعرّف `FrontendParser.g4` **28 قاعدة نحوية** قاعدة البداية فيها `document`، وتتوزّع على اللغات الثلاث:

اللغة	الرموز المخصّصة	القواعد النحوية
HTML	DOCTYPE TAG_OPEN TAG_CLOSE TAG_SLASH STYLE_TAG EQUALS TEXT HTML_COMMENT	document · content · contentItem · htmlElement · tagName · htmlAttribute · attrName · attrValue · text
CSS	CSS_LBRACE CSS_RBRACE CSS_COLON CSS_SEMICOLON CSS_UNIT CSS_IDENT CSS_COMMENT HASH	styleElement · cssRule · cssSelector · cssSelectorPart · cssSelectorItem · cssDeclaration · cssPropertyName · cssValue

اللغة	الرموز المخصصة	القواعد النحوية
Jinja2	3 محدّدات في النمط الافتراضي + 33 قاعدة داخل نمط JINJA	<pre> jinjaExpression · jinjaExpr · jinjaStatement · jinjaIfStatement · jinjaElifStatement · jinjaElseStatement · jinjaForStatement · jinjaExtendsStatement · jinjaBlockStatement · jinjaCondition · jinjaStmntExpr </pre>

يُلاحظ أن CSS **محلّة تحليل بنيوياً حقيقياً** إلى قواعد ومحدّدات وتصريحات، وليست نصّاً خاماً مبتلأً. وقد خُصص لها رمزان لافتان:

```

CSS_IDENT : [a-zA-Z] [a-zA-Z0-9]* ('-' [a-zA-Z0-9]+) ; // one token, not IDENTIFIER MINUS
IDENTIFIER
CSS_UNIT  : 'px' | '%' | 'em' | 'rem' ; // cssValue : NUMBER CSS_UNIT?

```

فبفضل CSS_IDENT تصل الخاصية background-color رمزاً **واحدًا** بدل ثلاثة رموز IDENTIFIER MINUS IDENTIFIER .

2.5 نمط المحلّل اللفظي JINJA

تواجه اللغة الأمامية مشكلة **تصادم الكلمات المفتاحية**: الكلمات if و for و in و block و and كلمات مفتاحية في Jinja2، لكنها في الوقت نفسه كلمات إنكليزية عادية قد ترد في نصّ HTML.

الحل نمط JINJA : لا تُعرّف الكلمات المفتاحية إلا **داخل** هذا النمط، ولا يُدخّل النمط إلا عبر المحدّدين {% و {{ :

```

JINJA_EXPR_START : '{{' -> pushMode(JINJA) ;
JINJA_STMT_START : '{%' -> pushMode(JINJA) ;

mode JINJA;
JINJA_EXPR_END : '}}' -> popMode ;
JINJA_STMT_END : '%}' -> popMode ;
JINJA_IF      : 'if' ;
JINJA_FOR     : 'for' ;

```

فتبقى الكلمة for خارج المحدّدات نصّاً عادياً يطابق IDENTIFIER ، وتصبح داخلها كلمة مفتاحية.

تحقق عملي. تحليل السطر {% for p in products %}{{ p.name }}{% endfor %} يُنتج التسلسل:

```

JINJA_STMT_START · JINJA_FOR · IDENTIFIER · JINJA_IN · IDENTIFIER
JINJA_STMT_END
JINJA_EXPR_START · IDENTIFIER · DOT · IDENTIFIER · JINJA_EXPR_END
JINJA_STMT_START · JINJA_ENDFOR · JINJA_STMT_END

```

فالكلمات المفتاحية لم تظهر إلا داخل المحدّدات.

2.6 معالجة ملف CSS المستقل

ملف `style.css` لا يحوي وسم `<style>`، فلا سبيل للوصول إلى القاعدة `styleElement`. عالج المترجم ذلك ببناء الوسم اصطناعياً قبل التحليل:

```
public CompilationResult compileCss(String filePath) {
    String css = readFile(filePath);
    return compileFrontendSource(filePath, "<style>" + css + "</style>");
}
```

ولأن الالتفاف لا يُدرج أي سطر جديد، تبقى **أرقام الأسطر مطابقة** لأرقام الملف الأصلي - فأول قاعدة `body` تُبلّغ عند [Line 1].

2.7 كشف الأخطاء النحوية

استُبدل المستمع الافتراضي لـ ANTLRJ بمستمع خاص يوحد صيغة الرسائل:

```
private void attachErrorListener(Parser parser) {
    parser.removeErrorListeners();
    parser.addErrorListener(new BaseErrorListener() {
        @Override
        public void syntaxError(Recognizer, ? recognizer, Object offendingSymbol,
                                int line, int charPositionInLine, String msg,
                                RecognitionException e) {
            System.err.println("    ERROR at line " + line + ":"
                               + charPositionInLine + " - " + msg);
        }
    });
}
```

ويؤخذ **عدد الأخطاء** من `parser.getNumberOfSyntaxErrors()` : فإن كان غير صفري لا يُشغّل زائر بناء الشجرة أصلاً، ويُنهى البرنامج بالحالة 1. وهذا خرج حقيقي على الملف `def broken(:`

```
ERROR at line 1:11 - no viable alternative at input 'defbroken(:'
FAILED with 1 syntax error(s)
```

2.8 قياسات المرحلة

الملف	الرموز	عقد شجرة الاشتقاق	AST عقد الـ
run_app.py	410	821	173
القوالب الستة	194 - 431	—	50 - 130
style.css	982	1,693	259
المجموع	3,321	6,185	957

بصفر أخطاء نحوية.

3 بناء الشجرتين والتابع المولّد

المطلب الثاني: بناء شجرتين مجرّدتين، الأولى لـ Python والثانية لـ Jinja2، مع تابع مولّد (Generator) يمرّر البيانات من مصفوفة البيانات في كود Python إلى الشجرة الثانية.

3.1 نمط الزائر (Visitor Pattern)

تُبنى كل شجرة عبر زائر يرث من الصنف الذي يولّده ANTLR تلقائياً، ويعيد تعريف تابع زيارة واحد لكل قاعدة نحوية، مُعيداً عقدة مكتوبة يدوياً:

```
public class BackendASTVisitor extends PythonFlaskParserBaseVisitor<ASTNode> {

    @Override
    public ASTNode visitForStatement(PythonFlaskParser.ForStatementContext ctx) {
        int line = ctx.start.getLine();
        String iterVar = ctx.IDENTIFIER().getText();
        ForStatementNode forNode = new ForStatementNode(line, iterVar);
        forNode.addChild(visit(ctx.expression()));
        forNode.addChild(buildBlock(line, "body", ctx.statement()));
        return forNode;
    }
}
```

الزائر	التوابع المتاحة	المُعاد تعريفها	ملاحظة
BackendASTVisitor	43	34	يملأ جدول الرموز أثناء المرور
FrontendASTVisitor	38	17	بنويّ محض

3.2 حجم الشجرتين

الشجرة	عدد العقد
شجرة Python (run_app.py)	173
شجرة Jinja (ستة قوالب)	محلّة + 9 محقونة = 534

وتفصيل القوالب: base · 50 · products · 130 · add · 105 · edit · 90 · detail · 81 · delete · 78.

3.3 التابع المولّد Generator

هذا هو جوهر المطلب الثاني. يعمل الصنف `generator.Generator` على ثلاث خطوات:

الخطوة الأولى – العثور على مصفوفة البيانات في الشجرة الأولى

يُمسح التابع `collectDataArrays` الشجرة بحثاً عن إسنادات على مستوى الوحدة تكون قيمتها قائمةً من القواميس، فيلتقط `products = [...]`.

الخطوة الثانية – قراءة الربط بين المتغيرات

يقرأ التابع `readRenderTemplate` كل استدعاء `render_template` ليعرف أي متغير في القالب يُغذى بأي متغير في Python:

```
return render_template('products.html', products=products, total=total)
```

فيستنتج أن القالب `products.html` يتلقى المتغير `products` من المتغير `products`. ولأن الربط يُقرأ من **الوسائط المسماة** لا من تطابق الأسماء، فإن إعادة التسمية مثل `items=products` تعمل كذلك.

الخطوة الثالثة – الحقن في الشجرة الثانية

يمرّ المولّد على شجرة Jinja ويضيف عقداً حقيقية من النوع `JinjaBoundDataNode` أبناءاً للعقد المعنية.

3.4 نتيجة الحقن

على التطبيق النموذجي نُحقّن **تسع عقد** كلها في `products.html`:

النوع	العدد	المحتوى
علامات تكرار	3	<code>product[0]</code> · <code>product[1]</code> · <code>product[2]</code>
تعبير محلولة	6	ثلاث قيم لـ <code>product.name</code> وثلاث لـ <code>product.details</code>

وتظهر هذه العقد **فعلياً** في طباعة الشجرة. وهذا مقطع حقيقي من خرج المرحلة الرابعة:

```
#340 [Line 35] JinjaExpression: {{ product.name }}
#1143 [Line 35] JinjaBoundData: product.name = Ali Yazbek
#1146 [Line 35] JinjaBoundData: product.name = Ali Suliman
#1149 [Line 35] JinjaBoundData: product.name = Sara Nabhan
```

حقن قيم مصفوفة Python كعقد داخل الشجرة الثانية

كما تظهر في الملف `compiler_output/ast_jinja.json`، فالبيانات صارت **جزءاً من الشجرة الثانية** لا مجرد جدول جانبي.

3.5 التمييز بين `Generator` و `ContextBuilder`

يتضمّن المشروع مساراً ثانياً للبيانات يجب تمييزه عن الأول:

الصف	الوظيفة	الناتج
Generator	يوثّق الربط بحقن عقد في الشجرة	شجرة معنونة تُظهر الربط عند الطباعة
ContextBuilder	يقيم البيانات إلى قيم فعلية	كائن <code>TemplateContext</code> يُستهلك في التوليد

فالأول يخدم **الإظهار** كما يطلب المطلب الثاني، والثاني يخدم **التوليد** كما يطلب المطلب الخامس. ويمثّل `TemplateContext` القيم بنماذج حقيقية – قوائم وقواميس وقيم مفردة – لأن قاموساً مسطّحاً من السلاسل النصية لا يكفي لدعم حلقة `{% for %}`.

المطلب الثالث: تطبيق مفاهيم البرمجة الكائنية – الوراثة (Inheritance) وتعدد الأشكال (Polymorphism) – مع تخزين معلومات كل عقدة، رقمها ورقم سطرها، بشكل منفصل لكل عقدة.

4.1 الصنف الأساس ASTNode

تتحدث كل عقدة في المشروع من صنف مجرد واحد يحمل أربعة حقول ويصرّح عن تابع مجرد واحد:

```
public abstract class ASTNode {

    private static final AtomicInteger NODE_COUNTER = new AtomicInteger();

    protected final int nodeId;
    protected String nodeName;
    protected int lineNumber;
    protected List<ASTNode> children;

    public ASTNode(String nodeName, int lineNumber) {
        this.nodeId = NODE_COUNTER.incrementAndGet();
        this.nodeName = nodeName;
        this.lineNumber = lineNumber;
        this.children = new ArrayList<>();
    }

    public abstract String getNodeInfo();
}
```

4.2 رقم العقدة ورقم السطر

رقم العقدة يُخصّص في بانية الصنف الأساس من عدّاد ذري (AtomicInteger)، فيكون الترقيم **عالمياً ومتزايداً** عبر اللغتين معاً، ويُضمن تفرّده حتى مع تعدد الخيوط.

رقم السطر يُمرّر إلى البانية نفسها من سياق ANTLR عبر `ctx.start.getLine()` – وهو استدعاء يتكرر ثماني وعشرين مرة في الزائر الخلفي وسبع عشرة مرة في الأمامي، أي عند كل بناء عقدة.

وهذا تحقّق عددي من ملفّي الخرج بعد ترجمة كاملة:

المؤشر	القيمة
عقد شجرة Python	173
عقد شجرة Jinja	534
المجموع	عقدة 707 • رقماً 707
عقد بلا رقم أو بسطر يساوي صفراً	0
تقاطع الأرقام بين الشجرتين	0

فكل عقدة في المشروع تحمل رقمها وسطرها، ولا يتكرر رقم واحد.

4.3 مخطط بنية الشجرة (شجرة الوراثة)

يضم المشروع 37 صنفاً: صنفان مجردان و35 صنفاً ملموساً، موزعة على مستويين من الوراثة.



مخطط 1 - بنية شجرة القواعد المجردة: صنف أساس مجرد، وصنف وسيط مجرد، و35 صنفاً ملموساً

4.4 الحقول الخاصة بكل عقدة

يضيف كل صنف الحقول التي تحمل معناه وحده، وكلها `private final` ومكتشفة بتوابع قراءة. وهذه أبرزها:

الصنف	الحقول المضافة فوق الأربعة الأساسية
FunctionDefNode	functionName · parameters : List<String>
AssignmentNode	variableName
ForStatementNode	iteratorVariable

الصف	الحقول المضافة فوق الأربعة الأساسية
BinaryOpNode	operator
LiteralNode	type · value
HtmlElementNode	isSelfClosing : boolean · tagName
JinjaForNode	collectionExpression · iteratorVariable
JinjaBranchNode	condition · keyword
JinjaBoundDataNode	resolvedValue · binding
CssDeclarationNode	value · property

4.5 تعدد الأشكال (Polymorphism)

التابع `getNodeInfo()` مجرّد في الصف الأساس. فيلزم كل صف ملموس بوصف نفسه: **35 من 35 صنفاً** تعيد تعريفه. أي مئة بالمئة.

والأهم أن **موضع الاستدعاء واحد** – في الصف الأساس عبر مرجع من النوع `ASTNode` – بينما يختلف السطر المطبوع تبعاً للنوع الفعلي للكائن:

```
// ForStatementNode
return String.format("[Line %d] %s: for %s in %s",
    lineNumber, nodeName, iteratorVariable, collectionExpression);

// HtmlElementNode
return String.format("[Line %d] %s: <%s>...</%s>",
    lineNumber, nodeName, tagName, tagName);

// JinjaBoundDataNode
return String.format("[Line %d] %s: %s = %s",
    lineNumber, nodeName, binding, resolvedValue);
```

فاستدعاء واحد `child.printTree(indent + 1)` يُنتج سطرًا مختلفاً لكل نوع عقدة – وهذا هو **الربط المتأخّر (Dynamic Dispatch)** بعينه: الصف الأساس لا يعرف أنواع أبنائه. ومع ذلك يستدعي سلوكها الصحيح.

خلاصة المطلب الثالث. الوراثة منقّقة على مستويين (`ASTNode` ثم `ExpressionNode`). وتعدد الأشكال متحقّق بتابع مجرّد يُعاد تعريفه في 35 صنفاً ويُستدعى من موضع واحد. ومعلومات كل عقدة – رقمها وسطرها – مخزّنة في العقدة نفسها لا في جدول خارجي.

المطلب الرابع: معالجة الأخطاء الدلالية في كلا الجزئين، بما لا يقل عن خمسة أخطاء.

5.1 الفحوص التسعة

عالج المشروع **تسعة أخطاء دلالية** موزعة على الجزئين – أي ما يقارب ضعف الحد الأدنى المطلوب. ويعرض الجدول التالي كل فحص، وجانبه، وخطورته، ومثالاً يُشغله فعلاً:

#	الرمز	الجزء	الخطورة	مثال يُشغله	ما يكشفه
1	undefined-name	Python	ERROR	<code>v=mystery</code> ولا إسناد لـ <code>mystery</code>	متغير يُستعمل ولم يُسند له شيء
2	undefined-function	Python	ERROR	<code>return calculate_total()</code>	استدعاء دالة غير معرّفة
3	missing-template	Python	ERROR	<code>render_template('nope.html')</code>	عرض قالب غير موجود
4	route-returns-nothing	Python	WARNING	دالة جسمها <code>x = 1</code> فقط	دالة عرض بلا <code>return</code>
5	undefined-template-variable	القوالب	ERROR	<code>{{ x }}</code> ولا مسار يمرّر <code>x</code>	متغير لا يمرّره أي مسار
6	unknown-field	القوالب	ERROR	<code>{{ product.titel }}</code>	حقل غير موجود في البيانات
7	not-iterable	القوالب	ERROR	<code>{% for x in total %}</code> و <code>total</code> عدد	حلقة على قيمة ليست قائمة
8	missing-layout	القوالب	ERROR	<code>{% extends 'no_such.html' %}</code>	وراثه من قالب غير موجود
9	unrendered-template	القوالب	WARNING	قالب منسيّ لا يعرضه مسار	قالب لا يعرضه أي مسار

أربعة في جانب Python · خمسة في جانب القوالب · سبعة أخطاء ومحذوران

5.2 مثال على رسالة تشخيصية

تحمل كل رسالة الملف ورقم السطر والرمز ونصاً موجزاً، مع اقتراح للاسم الصحيح عند الاقتضاء. ويعمل الفحص الخامس حتى داخل **قيم سمات HTML**. إذ تصل هذه القيم في الشجرة نصاً غير محلّل، فأُفرد لها ماسح خاص هو `checkAttributeValue`.

5.3 مكدّس النطاقات المستقل

يحتفظ المحلّل بمكدّس نطاقات خاص به من النوع `Deque<Map<String,Integer>>` بدل إعادة استعمال جدول الرموز. لأن مؤشّر الجدول **يعود إلى النطاق العام** فور انتهاء بناء الشجرة فلا يعود صالحاً لفحص يجري بعده. وتُمشى التعليمات **بترتيب ورودها**، ويُفحص الطرف الأيمن للإسناد **قبل** ربط الاسم – وهذا ما يجعل الاستعمال قبل الإسناد قابلاً للكشف.

5.4 رفع أسماء الدوال (Hoisting)

تُصرّح أسماء الدوال جميعها في **مرور تمهيدي أول** قبل الفحص، لأن Python تحلّ الاستدعاء وقت التنفيذ لا وقت القراءة – فيجوز لمسار أن يستدعي مساراً معرّفاً بعده في الملف دون أن يكون ذلك خطأً.

5.5 اقتراح الاسم الصحيح

عند وقوع خطأ إملائي يُقترح أقرب اسم متاح، بمسافة تحرير **Damerau-Levenshtein** لا بالمسافة التقليدية:

الخوارزمية	(title ← titel) تكلفة تبديل حرفين
Levenshtein التقليدية	حذف ثم إضافة - 2
Damerau-Levenshtein	تبديل واحد - 1

والفرق حاسم: عتبة الاقتراح لاسم من أربعة أحرف هي 1، فلو استُعملت الخوارزمية التقليدية لأعطت 2 ولضاع الاقتراح. وبما أن قلب حرفين أشيع الأخطاء الإملائية عند الإنسان، فإن الخوارزمية المختارة تُبقي الاقتراح حيّاً حيث تُسقطه الأخرى.

5.6 أثر الخطأ الدلالي على النتيجة

يجمع `app.Main` عدد الأخطاء الدلالية إلى إجمالي أخطائه، فينقلب الحكم النهائي إلى `COMPILATION FAILED` وتصبح حالة الخروج 1 بدل 0. وهذا خرج حقيقي بعد إدخال خطأ إملائي مقصود في أحد القوالب:

```
---- PHASE 6: SEMANTIC ANALYSIS (error checking) -----
ERROR product_detail.html:4  {{ product.titel }} - 'titel' is not a field of 'product'
ERROR product_detail.html:21  {{ product.titel }} - 'titel' is not a field of 'product'
ERROR product_detail.html:24  {{ product.titel }} - 'titel' is not a field of 'product'

3 error(s), 0 warning(s).
=====
COMPILATION FAILED - 3 error(s) found.
=====
```

المحلّل يكشف الخطأ الإملائي في ثلاثة مواضع ويحدّد سطر كل منها

5.7 التغطية الاختبارية

يغطّي هذه المرحلة **17 اختباراً** من أصل 64، وهي أكبر حصة اختبارية لأي مرحلة، وتنقسم إلى:

- **عشرة** اختبارات إيجابية تتحقق من إشعال الفحوص فعلاً؛
- **ستة** اختبارات تحرس من **الإيجابيات الكاذبة** – تتأكد أن الكود السليم لا يُبلّغ عنه؛
- **واحد** قياسي على التطبيق النموذجي.

فالمحلّل الذي ينذر بلا سبب أسوأ من غياب المحلّل، ولذلك خُصّص ثلث اختبارات المرحلة لحراسة هذه الناحية. وعلى التطبيق النموذجي يُبلّغ المحلّل عن **صفر ملاحظات**، أي أن التطبيق سليم دلاليّاً.

المطلب الخامس: أن تكون الأجزاء المولدة قادرة على العمل معاً.

6.1 المسار

```
run_app.py → ContextBuilder → TemplateContext (products=[...], total=3)
|
templates/*.html → Jinja AST → HtmlEmitter → output/*.html
```

يقرأ `ContextBuilder` شجرة Python ويطوي منها القيم الفعلية: يحلّ `len(products)` إلى 3، ويفهرس `products[index]` ويربط وسائط المسارات مثل `<int:id>`. ثم يعيد `HtmlEmitter` المرور على شجرة Jinja بهذا السياق ويُنتج HTML جاهزاً، ويكتبه `OutputWriter`.

6.2 معالجة بنى Jinja2

البنية	المعالجة
<code>{{ تعبير }}</code>	يُحلّ بالبحث في السياق مع هروب محارف HTML: وتعمل الاستبدالات أيضاً داخل قيم السمات، وهي حالة تسلمها القواعد رمزاً نصياً واحداً
<code>{% for %}</code>	يُشتق سياق فرعي جديد بـ <code>ctx.copy()</code> لكل صف، فيُعاد بثّ الجسم بقيم مختلفة في كل دورة
<code>{% if %} / {% elif %} / {% else %}</code>	يُحفظ كل فرع عقدةً مستقلة من النوع <code>JinjaBranchNode</code> ، ويعود التابع <code>emitIf</code> فور مطابقة أول فرع، فيُيث فرع واحد بالضبط
<code>{% extends %}</code> و <code>{% block %}</code>	تُدجّ كتل الابن في القالب الأب، فيغلب تعريف الابن عند تطابق الاسم

وفرع `else` لا يحمل شرطاً، ولذلك يطابق دائماً ويكون **المُتراجَع إليه** حين تفشل كل الفروع السابقة.

6.3 المولّد مقابل المنسوخ

الملفات	المعالجة	السبب
القوالب	تُولّد	هي موضوع الترجمة
<code>app.py</code> و <code>static/</code>	تُنسخ حرفياً	مُدخلات للتطبيق العامل، لا موضوعاً للترجمة

تسمية الصفحات تتبع المسار الذي يعرضها، فالمسار `'/'` يُنتج `index.html`. أما القالب `base.html` فيُستعمل بنية أساسية ولا يعرضه أي مسار، فلا يُنتج صفحة، ويُسجّل ذلك صراحةً في سجل التوليد.

6.4 قياسات التوليد

المؤشر	القيمة
الصفحات المولدة	بمجموع 9,728 بايت - 5

المؤشر	القيمة
الملفات المنسوخة	6
تعبير <code>{{ }}</code> محلولة / غير محلولة	35 / 0
دورات التكرار المنفّذة	3
فروع <code>{% if %}</code> مأخوذة / متروكة	3 / 1

وُكُتِبَ إلى جانب ذلك أربعة ملفات في `compiler_output/` : `ast_python.json` و `ast_jinja.json` و `semantic_report.txt` و `generation_log.txt`.

6.5 الخادم الحي

يُستبدل المجلد `src/server/` إطارَ Flask وقت التشغيل بأربعة أصناف:

الصف	الدور
<code>DataStore</code>	مصدر الحقيقة المتغيّر، يُبذّر مرة من مصفوفة Python ثم يملك البيانات
<code>SiteRenderer</code>	يعيد التوليد من محتوى المخزن الحالي
<code>AppServer</code>	خادم <code>com.sun.net.httpserver</code> يجيب المسارات وينفّذ التعديلات
<code>SourceWatcher</code>	يراقب <code>run_app.py</code> والقوالب، فيعيد الترجمة عند أي تعديل

ولماذا وُجِدَ المخزن؟ لأن `ContextBuilder` يقرأ البيانات من الشجرة، وهي ثابتة وقت الترجمة. فلو أُعيد التوليد من الشجرة وحدها لُنتج صفحاتٍ متطابقة مهما فعل الزائر. ولا يصبح إعادة التوليد ذا معنى إلا حين يملك مخزّن متغيّر البيانات. والقيم **المشتقة** يُعاد اشتقاقها كذلك: فالعدد `total` جاء من `len(products)`، ولذلك يُعاد حسابه بعد كل تعديل بدل الاحتفاظ بالقيمة التي كانت صحيحة وقت الترجمة.

المطلب السادس: تصميم واجهات ويب متكاملة – عرض المنتجات، وإضافة، وعرض التفاصيل، والحذف – مع تنقل سلس بينها.

7.1 جدول المسارات

المسار	الطريقة	الدالة	القالب	الوظيفة
/	GET	display_products	products.html	عرض القائمة
/add	GET · POST	add_product	add_product.html	إضافة عضو
/product/<int:id>	GET	product_detail	product_detail.html	عرض التفاصيل
/edit/<int:id>	GET · POST	edit_product	edit_product.html	تعديل عضو
/delete/<int:id>	GET	delete_product	delete_confirm.html	تأكيد الحذف
/confirm-delete/<int:id>	POST	confirm_delete	–	تنفيذ الحذف

وينفذ `AppServer` المسارات نفسها: خمسة مسارات `GET` مع احتياطي للملفات الساكنة، وثلاثة مسارات `POST`، ويجب كل تعديل بإعادة توجيهه 303 بعد إعادة توليد الموقع.

7.2 مخطط التنقل

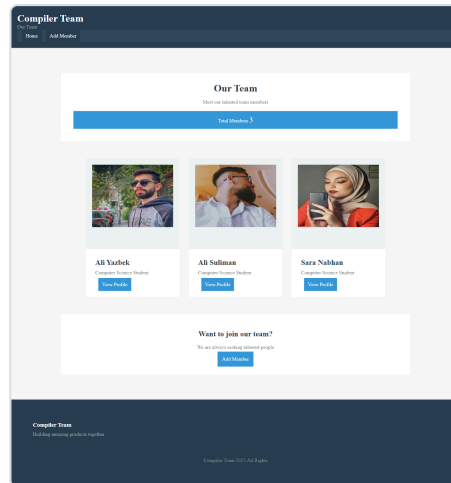
تحمل الصفحات الخمس **شريط تنقل موحدًا** يضم رابط الرئيسية ورابط الإضافة، فقائمة الأعضاء تُبلغ من أي صفحة بنقرة واحدة، وصفحة التفاصيل هي محور العمليات: منها يُنتقل إلى التعديل والحذف.



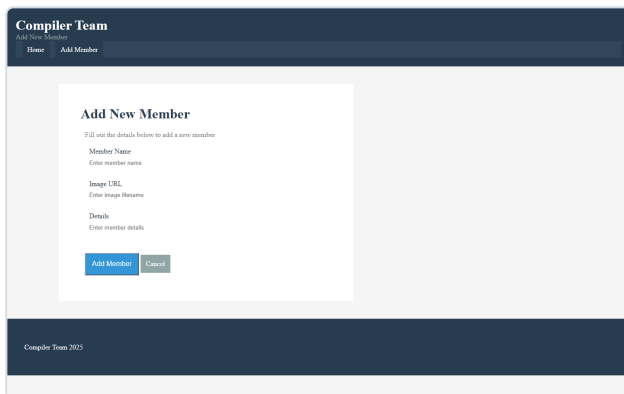
7.3 التصميم

الملف `style.css` مكتوب يدوياً في 313 سطراً تضم **57 قاعدة** و**143 تصريحاً**. بلوحة ألوان أساسها الكحلي `#2c3e50` والأزرق `#3498db` على أرضية فاتحة. وقد التزم فيه بمجموعة CSS التي تقبلها القواعد النحوية، فهو يُترجم بدوره في المرحلة الثالثة – 982 رمزاً و259 عقدة – ثم يُنسخ إلى `output/static/`.

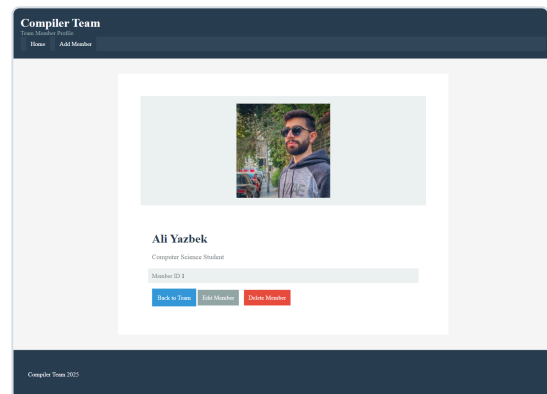
7.4 الواجهات المنقّدة



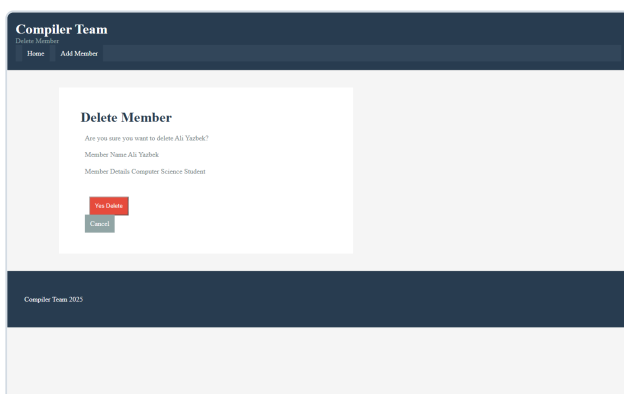
صورة 1 – قائمة الأعضاء (`GET /`): البطاقات الثلاث بالصور والأسماء، وشريط العدد الإجمالي المحسوب من `len(products)`.



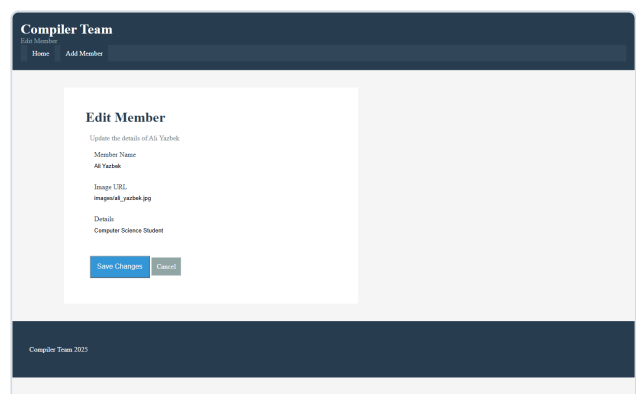
صورة 3 – استمارة الإضافة (`/add`).



صورة 2 – تفاصيل العضو (`GET /product/{id}`)، ومنها التعديل والحذف.



صورة 5 – تأكيد الحذف (`GET /delete/{id}`) قبل التنفيذ.



صورة 4 – استمارة التعديل (`/edit/{id}`) بحقولها المعبأة مسبقاً.

المطلب السابع: كتابة توابع لطباعة كل عقدة وأبنائها بشكل مقروء، وتوابع لطباعة الشجرة كاملة مع جدول الرموز.

8.1 طباعة العقد

عُرِّفت الطباعة مرة واحدة في الصنف الأساس، فورثتها كل العقد:

```
public abstract String getNodeInfo();

public String describe() {
    return String.format("#%-4d %s", nodeId, getNodeInfo());
}

public void printTree(int indent) {
    for (int i = 0; i < indent; i++) {
        System.out.print(" ");
    }
    System.out.println(describe());
    for (ASTNode child : children) {
        child.printTree(indent + 1);
    }
}
```

فالتابع `printTree` يطبع مسافتين لكل مستوى عمق، ثم سطر العقدة، ثم يستدعي نفسه على الأبناء بعمق أكبر بواحد. والتابع `describe` يصدر السطر برقم العقدة. أما `getNodeInfo` فمجرد، فيلزم كل صنف بوصف نفسه – و35 صنفاً من 35 تفعل ذلك.

8.2 مقطع حقيقي من طباعة الشجرة

```
#340 [Line 35] JinjaExpression: {{ product.name }}
#1143 [Line 35] JinjaBoundData: product.name = Ali Yazbek
#1146 [Line 35] JinjaBoundData: product.name = Ali Suliman
#1149 [Line 35] JinjaBoundData: product.name = Sara Nabhan
#341 [Line 36] HtmlElement: <p>...</p>
#342 [Line 36] HtmlAttribute: class="productdetails"
#344 [Line 36] JinjaExpression: {{ product.details }}
#1147 [Line 36] JinjaBoundData: product.details = Computer Science Student
#345 [Line 37] HtmlElement: <div>...</div>
```

يقرأ من هذا المقطع أربعة أشياء دفعة واحدة:

- **رقم العقدة** بادئة لكل سطر – وهو مميّز على مستوى المشروع كله;
- **رقم السطر** المصدري الذي جاءت منه العقدة;
- **نوع العقدة** ثم الخصائص التي تحمل معناها وحدها;
- **الإزاحة** التي تُظهر علاقة الأبوة والبنوة مباشرة.

8.3 جدول الرموز

يُطبع جدول الرموز بصورتين متكاملتين. الأولى جدول من خمسة أعمدة مرتّب حسب النطاق ثم رقم السطر:

```

=== Symbol Table ===
Name                Type                Data Type      Line   Scope
=====
method              variable            unknown        18     add_product
name                 variable            unknown        19     add_product
new_product          variable            dict           23     add_product
success              variable            string         25     add_product
id                   parameter           unknown        47     confirm_delete

```

والثانية شجرة النطاقات المطبوعة تعاودياً بإزاحة تُظهر التداخل:

```

=== Scope Tree ===
Scope: global
- Flask (import, module)
- render_template (import, module)
- app (variable, unknown)
- products (variable, list)
- display_products (function, function)
- add_product (function, function)

```

وعلى التطبيق النموذجي يُنتج المترجم **32 رمزاً** موزعة على **سبعة نطاقات**: النطاق العام وستة نطاقات للدوال.

8.4 الطباعة بصيغة JSON

توفّر صورة ثانية للطباعة قابلة للمعالجة الآلية عبر `AstJsonSerializer`، تكتب لكل عقدة الحقول الأربعة `id` و `node` و `line` و `children`، مضافاً إليها خصائص كل نوع:

```

{
  "id": 2,
  "node": "ImportStatement",
  "line": 1,
  "module": "flask",
  "imports": ["Flask", "render_template", "request", "redirect"]
}

```

وتُنتج بذلك `ast_python.json` بمئة وثلاث وسبعين عقدة، و `ast_jinja.json` بستة قوالب تضم أربعمئة وأربعاً وثلاثين وخمسمئة عقدة.

9.1 حصيلة المطالب

المطلب	التحقيق
1 – Lexer & Parser للغات الأربع	4 ملفات قواعد · 475 سطرًا · 97 نوع رمز · 52 قاعدة نحوية · نمطان لفظيان
2 – شجرتان + مولّد	534 + 173 عقدة · 9 عقد بيانات محقونة في الشجرة الثانية
3 – بنية العقد الكائنية	37 صنفاً · وراثة على مستويين · 35 إعادة تعريف لـ <code>getNodeInfo</code> · رقم وسطر لكل عقدة
4 – التحليل الدلالي	9 فحوص على الجزئين · 17 اختباراً
5 – توليد الكود	5 صفحات عاملة · 35 تعبيراً محلولاً · 0 غير محلول
6 – الواجهات والتنقل	6 مسارات · 5 صفحات · شريط تنقل موحد
7 – الطباعة	طباعة كل عقدة وأبنائها · الشجرة كاملة · جدول الرموز وشجرة النطاقات

9.2 نطاق المترجم المدعوم

يغطي المترجم مجموعة جزئية محدّدة من كل لغة، اختيرت لتكون كافية لتطبيقات Flask النموذجية:

اللغة	البنى المدعومة
Python	الاستيراد · الإسناد · تعريف الدوال ووسائطها · المزخرفات (Decorators) · الشروط <code>if/elif/else</code> · الحلقات <code>for</code> · الإرجاع · القوائم والقواميس والاستدعاءات · أحد عشر عاملاً بأسبقياتها
Jinja2	التعابير <code>{{ }}</code> · الحلقات <code>{% for %}</code> · الشروط <code>{% if %}/{% elif %}/{% else %}</code> · الوراثة <code>{% extends %}</code> و <code>{% block %}</code>
HTML	العناصر والسمات والنصوص · العناصر ذاتية الإغلاق · التعليقات
CSS	القواعد والمحدّدات – بالأصناف والمعرّفات – والتصريحات والوحدات

9.3 آفاق التطوير

يمكن توسيع المشروع في اتجاهات عدة: دعم الإزاحة الدلالية (`DEDENT` / `INDENT`) في قواعد Python لتغطية البنى المتداخلة العميقة؛ وإضافة مرسّحات Jinja2 مثل `|upper`؛ وتوسيع قواعد CSS لتشمل `@media` والأصناف الزائفة مثل `:hover`؛ وإضافة فحوص دلالية على جانب CSS مثل كشف الأصناف المستعملة في HTML وغير المعرّفة في ملف الأنماط.